

Bare-Metal Observability Platform - Intern Onboarding Guide

What this platform does

This repo runs a full observability and reliability platform on a Linux host without Docker. Terraform installs/configures systemd services for Prometheus, Loki, Tempo, Grafana, Alertmanager, exporters, OpenTelemetry Collector, the demo API, and the DORA exporter.

The main goal is to move from "something is wrong" to "this endpoint/span/log line caused it" quickly.

How the services work together

```
test-api -> metrics -> Prometheus
test-api -> traces -> OpenTelemetry Collector -> Tempo
systemd journals -> OpenTelemetry Collector -> Loki
Prometheus alerts -> Alertmanager -> Slack #detrudr-alerts-demo
Grafana -> Prometheus + Loki + Tempo
```

Important URLs

- Grafana: <http://localhost:3000>
- Prometheus: <http://localhost:9090>
- Alertmanager: <http://localhost:9093>
- Test API: <http://localhost:8080>
- Loki API: <http://localhost:3100>
- Tempo API: <http://localhost:3200>
- Node Exporter: <http://localhost:9100>
- Blackbox Exporter: <http://localhost:9115>
- DORA Exporter: <http://localhost:9108>

Loki and Tempo are API backends. A browser 404 at '/' is normal. Use Grafana Explore.

Deploying

```
cp terraform/terraform.tfvars.example terraform/terraform.tfvars
```

Edit 'terraform/terraform.tfvars' and set the Slack webhook, Grafana password, and optional GitHub credentials.

Deploy:

```
make up
```

Destroy local services and install directories:

```
make down
```

Useful CLI commands

Show service status:

```
make status
```

Follow logs for the full stack:

```
make logs
```

Restart the full stack:

```
make restart
```

Run health checks:

```
make health
```

Inspect one service:

```
systemctl status test-api --no-pager  
journalctl -u test-api -f
```

Generate normal traffic:

```
curl http://localhost:8080/workout
```

Generate slow traffic:

```
curl "http://localhost:8080/workout?delay_ms=1200"
```

Generate an error:

```
curl "http://localhost:8080/workout?fail=true"
```

Grafana

Open Grafana:

```
http://localhost:3000
```

Use the credentials from 'terraform/terraform.tfvars'.

Grafana has three datasource types:

- Prometheus for metrics.
- Loki for logs.
- Tempo for traces.

Dashboard: Unified Observability

Use this first during incidents.

Panels:

- Request Rate
- Error Rate
- Latency p50/p95/p99
- Correlated Loki logs
- Recent Tempo traces

Workflow:

1. Find a metric spike.
2. Keep the same time range.

3. Inspect Loki logs.
4. Click the 'trace_id'.
5. Inspect the Tempo trace and identify the slow/failing span.

Dashboard: SLO & Error Budget

Use this to answer: are we meeting our reliability promise?

Panels:

- Availability SLI
- Latency SLI
- Success SLI
- Error budget remaining
- Fast and slow burn rate
- 7-day and 30-day compliance

If burn rate rises, move to the Unified Observability dashboard and follow 'runbooks/slo-burn.md'.

Dashboard: DORA Metrics

Use this to understand delivery performance.

Panels:

- Deployment Frequency
- DORA classification
- Lead Time for Changes
- Change Failure Rate
- Deployment outcomes
- MTTR

Live GitHub Actions data requires 'github_repository', 'github_token', and 'deployment_workflow_name' in Terraform variables. Without them, the exporter emits fallback demo data.

Dashboard: Node Exporter

Use this for host health.

Panels:

- CPU total and per-core
- Memory used/cached/available
- Disk I/O
- Network I/O
- Load averages

If CPU or memory alerts fire, compare this dashboard with latency/error panels in Unified Observability.

Dashboard: Blackbox Exporter

Use this for user-facing availability.

Panels:

- Uptime/downtime timeline
- HTTP response time p50/p90/p99
- SSL expiration days
- Probe success rate

This tells whether the service works from an external probe point of view, not just whether the process is running.

Prometheus

Open:

`http://localhost:9090`

Useful pages:

- '/targets'
- '/alerts'
- '/graph'

Useful PromQL:

```
up
sum(rate(http_requests_total[5m])) by (route, status)
sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, route))
```

Loki

Use Loki from Grafana Explore.

Queries:

```
{service_name="test-api"}  
{service_name="test-api"} | json  
{service_name="test-api"} | json | trace_id != ""
```

Tempo

Use Tempo from Grafana Explore or by clicking a 'trace_id' in Loki logs.

TraceQL:

```
{ resource.service.name = "test-api" }
```

Alertmanager and Slack

Alertmanager receives alerts from Prometheus and sends structured payloads to:

```
#detrudr-alerts-demo
```

Every alert should include:

- name
- severity
- service/instance
- current value
- dashboard link
- runbook link
- firing/resolved status

Runbooks

Runbooks live in 'runbooks/'.

Start with the runbook linked in the Slack alert. If no alert is firing but the service looks degraded, start with 'runbooks/slo-burn.md' or 'runbooks/instance-down.md'.

Practice flow for a new intern

1. Run 'make up'.
2. Open Grafana.
3. Run 'curl http://localhost:8080/workout'.
4. Run 'curl "http://localhost:8080/workout?delay_ms=1200"'

5. Open Unified Observability and find the latency spike.
6. Open Loki logs for the same time range.
7. Click a 'trace_id'.
8. Inspect the Tempo trace.
9. Open Prometheus '/targets'.
10. Read the SLO dashboard and explain the error budget.

Mental model

Prometheus tells you what changed.

Loki tells you what the services said.

Tempo tells you where request time was spent.

Grafana brings everything together.

Alertmanager gets the right information into Slack.